

01-stan-ode-forcing-function

September 23, 2026

1 Using forcing functions in ordinary differential equations (ODEs) in Stan

1.1 Overview

Ordinary differential equations (ODEs) are widely used for mathematical modeling of natural systems. A less common but critical ODE solver input is the incorporation of fixed temporal data representing a process whose underlying equation is unknown, called a forcing function. Forcing functions are more readily supported in lower-level programming languages than in higher-level, domain-specific programming languages (DSLs) like Stan that allow fewer mathematical primitives and are more complex to extend. The sections below explain how to use a forcing function with the widely used 2-equation predator–prey ODEs example, also called Lotka–Volterra.

1.2 Preamble

First, load the modules we will need for the entire exercise.

```
[1]: # Python builtin modules.
from argparse import Namespace
from collections import OrderedDict
import re

# 3rd party modules.
import arviz as az
import arviz_plots as azp
import cmdstanpy
from lets_plot import (
    LetsPlot, aes, as_discrete, facet_wrap, ggplot, geom_line, geom_point,
    geom_vline, labs, scale_y_log10,
)
import numpy as np
import pandas as pd
from patsy import dmatrices
from scipy.integrate import solve_ivp
import scipy.stats
import statsmodels.formula.api as smf

# Configuration.
```

```
az.rcParams["plot.backend"] = "matplotlib"
LetsPlot.setup_html()
```

```
/__w/stan-ode-forcing-function/stan-ode-forcing-
function/stanodeffpy/.venv/lib/python3.14/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm
/__w/stan-ode-forcing-function/stan-ode-forcing-
function/stanodeffpy/.venv/lib/python3.14/site-
packages/lets_plot/plot/annotation.py:592: SyntaxWarning: "\"(" is an invalid
escape sequence. Such sequences will not work in the future. Did you mean "\"\\("?
A raw string is also an option.
    .line(r'\\(R\\^2=\\)@..r2..')\\
/__w/stan-ode-forcing-function/stan-ode-forcing-
function/stanodeffpy/.venv/lib/python3.14/site-
packages/lets_plot/plot/annotation.py:644: SyntaxWarning: "\"(" is an invalid
escape sequence. Such sequences will not work in the future. Did you mean "\"\\("?
A raw string is also an option.
    .line(r'\\(R\\^2=\\)@..r2..')\\
```

```
[2]: %load_ext cmdstanjupyter
```

```
[3]: # One time Stan setup.
      assert cmdstanpy.install_cmdstan()
```

CmdStan install directory: /github/home/.cmdstan

Installing CmdStan version: 2.40.0

Downloading CmdStan version 2.40.0

Failed to download CmdStan version 2.40.0 from github.com

<urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self-
signed certificate (_ssl.c:1082)>

retry (1/5)

Download successful, file: /tmp/tmpvi9t9n5f

Extracting distribution

Unpacked download as cmdstan-2.40.0

Building version cmdstan-2.40.0, may take several minutes, depending on your
system.

Installed cmdstan-2.40.0

Test model compilation

The data science library **pandas** requires little introduction for many Python users.

Note: Those less familiar can learn about them from the freely available
Carpentries lesson *Programming with Python* (<https://swcarpentry.github.io/python-novice-inflammation/>) and, for a more depth, the excellent freely available
book *Python for Data Analysis* (<https://wesmckinney.com/book/>).

Note: The grammar of graphics implemented by `lets_plot` is probably more well known among R users, so users coming from `matplotlib`, `seaborn`, etc. may benefit from the freely available book *Python for Data Science* that provides a nice overview of the `lets-plot` Python library (<https://aeturrell.github.io/python4DS/data-visualise.html>).

While our primary goal is to use `stan` to solve the ODEs, we use `scipy.integrate.solve_ivp` to generate the *ground truth* data; ground truth data here refers to generating observations with known *true* parameters so that we can assess how well the fitted parameters compare with the true parameters.

Note: It is also possible to use Stan (`stan`) itself to generate the ground truth data, as described in the Stan User’s Guide > Example Models > Ordinary Differential Equations > Measurement error models. But for simplicity, we use Python’s popular `solve_ivp` function (that replaces the deprecated `odeint` function) to use familiar Python code where possible.

A well written Stan model should be relatively insensitive to initial conditions. The example below, demonstrates a case of undesirable sensitivity to initial conditions. **FIXME:** The `posterior` package simplifies inspecting the data wrangling work to visualize the ODE parameters accepted by Stan for each chain to troubleshoot disagreement among chains.

1.3 Typical ODEs without a forcing function

1.3.1 Equations in Python

$$\begin{aligned}\frac{dx}{dt} &= \alpha x - \beta xy \\ \frac{dy}{dt} &= -\gamma y + \delta xy\end{aligned}$$

where,

State variables:

x : Population of prey
 y : Population of predators

Parameters:

α : Growth rate of prey
 β : Death rate of prey due to predators
 γ : Death rate of predators
 δ : Growth rate of predators due to prey

To code these equations in Python, we need to wrap them in a function with specific function arguments common to many ODE solvers, namely, the time vector, the list of state variables, and then the list of parameters. Therefore, the input to `solve_ivp()` from the `scipy` package is written as:

```

[4]: true_y0 = OrderedDict((
    ("x", 10,),
    ("y", 5,),
))
true_params = OrderedDict((
    ("a", 1.0,),
    ("b", 0.5,),
    ("g", 1.5,),
    ("d", 0.7,),
))

def lv(t, y_, p=true_params):
    """Right-hand side of the Lotka-Volterra ODEs.

Parameters
    -----
    t : float
        Single timepoint at which to solve the system of equations.
    y_ : array_like
        State variables at the timepoint `t`.
    p : dict of float, optional
        Model parameters.

    """
    # Unpack y into convenience variances that must match the return value of
    # this function.
    x, y = y_

    # Decode the initial parameters and initial state variable values.
    p = Namespace(**p)

    # Encode the equation terms.
    prey_growth = p.a * x
    prey_death = p.b * x * y
    predator_growth = p.d * x * y
    predator_death = p.g * y

    # Return the equation RHS.
    dx_dt = prey_growth - prey_death
    dy_dt = predator_growth - predator_death
    return (dx_dt, dy_dt,)

```

The parameters are stored in an `OrderedDict` object to improve legibility by matching the equation order of parameters. The `Namespace` unpacks the parameters more conveniently with dot notation (`p.alpha`) instead of dictionary square brackets (`p["alpha"]`). However, in production code, some of these niceties may need to be omitted, especially if they significantly effect the ODE solver speed as benchmarked by Jupyter's `%timeit`, etc.

1.3.2 Generate noisy input data with known ODE parameters

A common statistical practice is to generate data with known parameters so that we can test how well a method or model is able to recover the parameters used to generate the data. Therefore, we next simulate ODE data using known parameters a , b , g , and d (i.e. α , β , γ , and δ). Additionally, we need to assume a known initial condition x_0 and y_0 stored in `true_y0`. Lastly, to make the parameter fitting more challenging and realistic to real-world observations, we add simulated noise.

```
[5]: # Solve the ODEs, then add noise to each variable using the Normal
# distribution.
times = np.linspace(0, 15, 31)
pristine = solve_ivp(fun=lv,
                     t_span=[np.min(times), np.max(times)],
                     y0=list(true_y0.values()),
                     t_eval=times)
# Check there was no solver error.
assert pristine.success
# Add noise to the pristine ODE solution.
noisy = pristine.y.copy()
prng = np.random.Generator(np.random.MT19937(seed=123))
noisy += prng.normal(size=noisy.shape, scale=0.5)
noisy[noisy < 0] = 0
```

Using `solve_ivp()` above only solves the system of equations at the timepoints specified by `times`. The simulated “pristine” and “noisy” data points are shown below, although from here on we will only use the noisy simulated data.

```
[6]: def ode_to_df(arr, time=pristine.t[:, np.newaxis]):
    return pd.DataFrame(np.concat([time, arr.T], axis=1),
                        columns=["time"] + list(true_y0.keys())[:arr.shape[0]])

data = (
    (
        pd.concat([ode_to_df(pristine.y), ode_to_df(noisy)],
                  keys=["pristine", "noisy"])
        .reset_index(level=0, names=["type"])
    )
    .reset_index(drop=True)
).reset_index(names="i")
data.groupby("type").head()
```

```
[6]:
```

	i	type	time	x	y
0	0	pristine	0.0	10.000000	5.000000
1	1	pristine	0.5	1.062272	12.656768
2	2	pristine	1.0	0.152750	6.922895
3	3	pristine	1.5	0.073187	3.386414
4	4	pristine	2.0	0.066596	1.637037

```

31 31    noisy  0.0   9.929955   5.418843
32 32    noisy  0.5   1.051534  11.932301
33 33    noisy  1.0   1.008484   7.074044
34 34    noisy  1.5   0.000000   3.488742
35 35    noisy  2.0   0.785979   1.449207

```

```
[7]: data = pd.melt(data, id_vars=["i", "type", "time"])
data
```

```

[7]:      i      type  time variable      value
0     0  pristine  0.0         x  10.000000
1     1  pristine  0.5         x   1.062272
2     2  pristine  1.0         x   0.152750
3     3  pristine  1.5         x   0.073187
4     4  pristine  2.0         x   0.066596
..    ..      ...  ...      ...      ...
119  57    noisy  13.0         y   0.000000
120  58    noisy  13.5         y   0.052869
121  59    noisy  14.0         y   0.000000
122  60    noisy  14.5         y   0.171035
123  61    noisy  15.0         y   0.000000

```

[124 rows x 5 columns]

```
[8]: (
    ggplot(data,
           aes("time", "value", color="variable")) +
    facet_wrap("type", ncol=1) +
    geom_line() +
    geom_point()
)
```

```
[8]: <lets_plot.plot.core.PlotSpec at 0x7fe03147e9e0>
```

1.3.3 Fit the ODE parameters using the equivalent Stan model

The model we are going to fit is saved in a separate `lv.stan` file that looks like this:

```
[9]: %stanf lv.stan model_lv
```

```
INFO:cmdstanjupyter:Creating CmdStanPy model & assigning it to variable
"model_lv"
```

```
20:50:15 - cmdstanpy - INFO - compiling stan file /__w/stan-ode-forcing-
function/stan-ode-forcing-function/stanodeffpy/lv.stan to exe file /__w/stan-
ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv
```

```
20:50:24 - cmdstanpy - INFO - compiled model executable: /__w/stan-ode-forcing-
function/stan-ode-forcing-function/stanodeffpy/lv
```

```
INFO:cmdstanjupyter:StanModel now available as variable "model_lv"!
Compilation took 8 seconds.
```

```
functions{
  vector lv(real t,
            vector y,
            real a, real b, real g, real d){
    vector[2] dydt;
    // y[1] and y[2] correspond to x and y respectively.
    dydt[1] = a*y[1] - b*y[1]*y[2];
    dydt[2] = -g*y[2] + d*y[1]*y[2];
    return dydt;
  }
}

data {
  int<lower=1> T;
  array[T] vector[2] y;
  vector<lower=0>[2] y0;
  real<lower=0> t0;
  array[T] real ts;
  real<lower=0> g;
  real<lower=0> d;
}

parameters {
  // y0 contains the initial values of x and y respectively.
  real<lower=0> a;
  real<lower=0> b;
}

model {
  array[T] vector[2] mu = ode_rk45(lv, y0, t0, ts, a, b, g, d);
  // Sample positive priors for all our ODE parameters.
  a ~ normal(0.5, 0.5);
  b ~ normal(0.5, 0.5);
  // The state variables computed by the ODE should be lognormally
  // distributed.
  for (t in 1:T) {
    y[t] ~ normal(mu[t], 1);
  }
}
```

```
[10]: fit1 = model_lv.sample(chains=4,
                             parallel_chains=4,
                             threads_per_chain=1,
                             iter_warmup=1_000,
                             iter_sampling=5_000,
```

```

        seed=123,
        show_progress=False,
        data={
            "T": noisy.T.shape[0] - 1,
            "y": noisy.T[1:],
            "y0": list(true_y0.values()),
            "t0": 0,
            "ts": pristine.t[1:],
            "g": true_params["g"],
            "d": true_params["d"],
        })

```

```

20:50:24 - cmdstanpy - INFO - CmdStan start processing
20:50:24 - cmdstanpy - INFO - Chain [1] start processing
20:50:24 - cmdstanpy - INFO - Chain [2] start processing
20:50:24 - cmdstanpy - INFO - Chain [3] start processing
20:50:24 - cmdstanpy - INFO - Chain [4] start processing
20:50:31 - cmdstanpy - INFO - Chain [3] done processing
20:50:40 - cmdstanpy - INFO - Chain [4] done processing
20:51:15 - cmdstanpy - INFO - Chain [2] done processing
20:54:36 - cmdstanpy - INFO - Chain [1] done processing

20:54:36 - cmdstanpy - WARNING - Non-fatal error during sampling:
Exception: ode_rk45: ode parameters and data is inf, but must be finite! (in
'lv.stan', line 30, column 2 to column 63)
    Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
    Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
    Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
    Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
    Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv.stan', line 30, column 2 to column 63)
    Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
    Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: ode_rk45: ode parameters and data is inf, but must be finite! (in
'lv.stan', line 30, column 2 to column 63)
    Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
    Exception: normal_lpdf: Location parameter[1] is -nan, but must be

```

```

finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be finite! (in
'lv.stan', line 37, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv.stan', line 30, column 2 to column 63)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: ode_rk45: ode parameters and data is inf, but must be finite! (in
'lv.stan', line 30, column 2 to column 63)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv.stan', line 30, column 2 to column 63)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv.stan', line 30, column 2 to column 63)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv.stan', line 37, column 4 to column 28)
Consider re-running with show_console=True if the above output is unclear!

```

```
[11]: print(fit1.diagnose())
```

```

Checking sampler transitions treedepth.
Treedepth satisfactory for all transitions.

```

```

Checking sampler transitions for divergences.
No divergent transitions found.

```

Checking E-BFMI - sampler transitions HMC potential energy.
 The E-BFMI, 0.00, is below the nominal threshold of 0.30 which suggests that HMC may have trouble exploring the target distribution.
 If possible, try to reparameterize the model.

The following parameters had fewer than 0.001 effective draws per transition:

a, b

Such low values indicate that the effective sample size estimators may be biased high and actual performance may be substantially lower than quoted.

The following parameters had rank-normalized split R-hat greater than 1.01:

a, b

Such high values indicate incomplete mixing and biased estimation.
 You should consider regularizing your model with additional prior information or a more effective parameterization.

Processing complete.

We get warnings about how our initial conditions were not able to agree and mix together that we will explore in the next section.

1.3.4 Inspect the Stan fit

```
[12]: ## Numerical summary of the fits.
fit1.summary()
```

```
[12]:
```

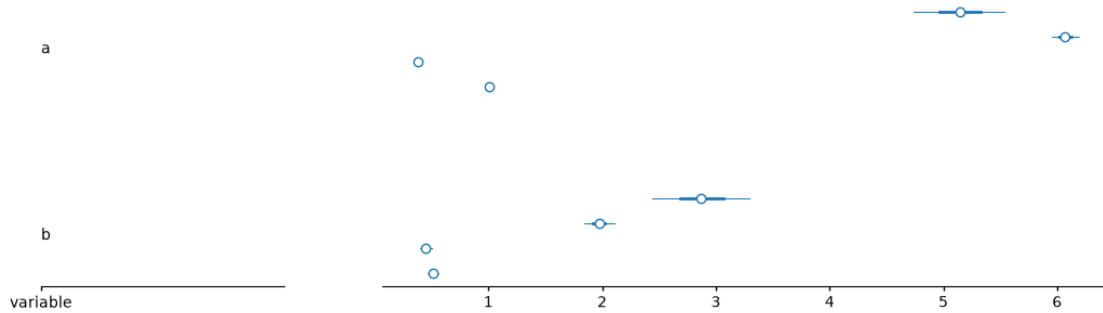
	Mean	MCSE	StdDev	MAD	5%	50% \
lp__	-302.29200	137.620000	194.68400	186.48300	-500.184000	-354.34200
a	3.14981	1.759540	2.49206	3.50618	0.367243	2.74267
b	1.45180	0.723586	1.03195	1.10875	0.417843	1.17091

	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-6.31331	4.58882	10000.0	0.017558	2.83913
a	6.13484	4.59230	10000.0	0.017571	2.83863
b	3.11131	4.72122	10000.0	0.018065	2.60661

```
[13]: ## Prepare data for plotting.
idata1 = az.from_cmdstanpy(fit1)
```

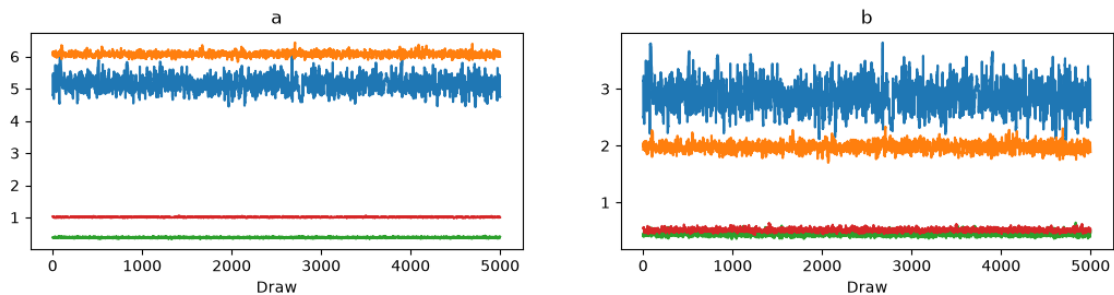
```
[14]: ## Parameter plot of the fits.
azp.plot_forest(idata1)
```

```
[14]: <arviz_plots.plot_collection.PlotCollection at 0x7fe0305138c0>
```



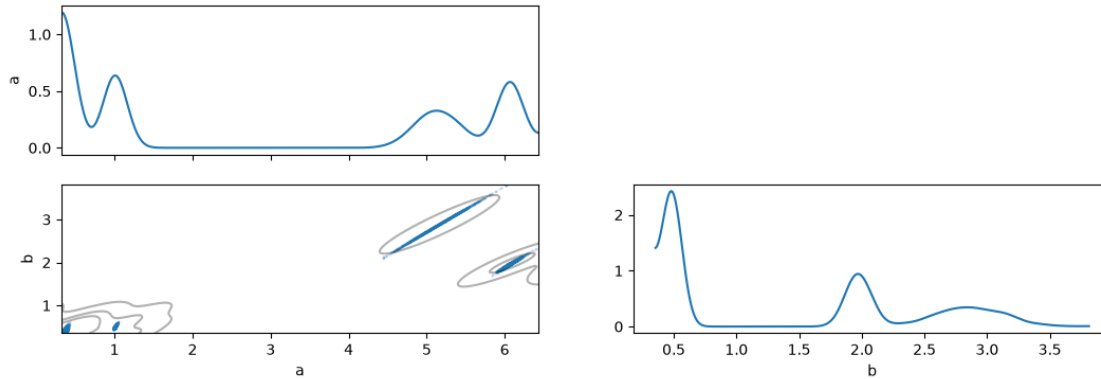
```
[15]: ## Show all samples after warmup.
azp.plot_trace(idata1)
```

```
[15]: <arviz_plots.plot_collection.PlotCollection at 0x7fe030641450>
```



```
[16]: ## Show the pairs plot.
## FIXME: Possible to include lp__?
azp.plot_pair(idata1,
              visuals={
                  "contour": {"color": "black", "alpha": 0.3},
                  "scatter": {"size": 2, "alpha": 0.1},
              })
```

```
[16]: <arviz_plots.plot_matrix.PlotMatrix at 0x7fe0306e4d70>
```



[17]: *## Plot the ODEs using a sample of 100 parameters from each of the chains.*

```
df_fit1 = fit1.draws_pd()
## Sample greater variation by only choosing one sample from each qcut.
```

```
qcut_20 = lambda x: pd.qcut(x, np.linspace(0, 1, 20), labels=False)
```

```
grouped = (
    df_fit1
    .filter(["chain__", "iter__", "a", "b"])
    .groupby("chain__")
)
```

```
n = 100
```

```
prng = np.random.Generator(np.random.MT19937(seed=123))
```

```
df_fit1_subset = (
    df_fit1.join(
        pd.DataFrame({
            "qa": grouped["a"].transform(qcut_20),
            "qb": grouped["b"].transform(qcut_20),
        })
    )
)
```

```
.groupby(["chain__", "qa", "qb"])
```

```
.sample(n=1, random_state=prng)
```

```
## Now sample the 100 parameters from each chain.
```

```
.groupby("chain__")
```

```
.sample(n=n, replace=True)
```

```
.reset_index()
```

```
.reset_index(names="sample")
```

```
)
```

```
n_node = df_fit1_subset.shape[0]
```

```
arr = np.empty(np.array(pristine.y.T.shape) * np.array([n_node, 1]),
               dtype=pristine.y.dtype)
```

```
for i in range(n_node):
```

```
    ode = solve_ivp(fun=lv,
```

```
                    t_span=[np.min(times), np.max(times)],
```

```
                    y0=noisy.T[0, :],
```

```

        t_eval=times,
        args=[
            {
                "a": df_fit1_subset.loc[i, "a"],
                "b": df_fit1_subset.loc[i, "b"],
                "g": true_params["g"],
                "d": true_params["d"],
            }
        ])
    assert ode.success
    arr[(i * len(times)):((i + 1) * len(times)), ] = ode.y.T
data = (
    pd.concat(
        (
            pd.DataFrame(
                {
                    "sample": np.repeat(range(n_ode), len(times)),
                    "time": np.tile(times, n_ode),
                }
            ),
            pd.DataFrame(arr, columns=true_y0.keys())
        ),
        axis=1
    )
    .join(df_fit1_subset.filter(["sample", "chain__"]),
        on="sample",
        lsuffix="l",
        rsuffix="r")
)
data = pd.melt(data, id_vars=["sample", "time", "chain__", "samplel",
↪ "sampler"])
data

```

```

[17]:
   sample  time  chain__  samplel  sampler  variable  value
0         0   0.0      1.0         0         0         x  9.929955
1         0   0.5      1.0         0         0         x  0.027550
2         0   1.0      1.0         0         0         x  0.004469
3         0   1.5      1.0         0         0         x  0.007298
4         0   2.0      1.0         0         0         x  0.035336
...
24795    399  13.0      4.0       399       399         y  0.031325
24796    399  13.5      4.0       399       399         y  0.019757
24797    399  14.0      4.0       399       399         y  0.015050
24798    399  14.5      4.0       399       399         y  0.015646
24799    399  15.0      4.0       399       399         y  0.026691

```

[24800 rows x 7 columns]

```
[18]: (
    ggplot(data, aes("time", "value", group=["sample", "variable"],
    ↪color="variable")) +
    facet_wrap("chain__") +
    geom_line()
)
```

```
[18]: <lets_plot.plot.core.PlotSpec at 0x7fe030a039b0>
```

In the last plot above that shows 100 ODE samples simulated by each of the 4 chains, we see that three of the chains are not able to capture the true periodicity. This is because of two reasons: (1) the oscillatory ODE system contains “peaks” of higher posterior probabilities and moving between these “peaks” is hard and (2) we are using large priors $a \sim \text{normal}(0.5, 0.5)$ and $b \sim \text{normal}(0.5, 0.5)$:

```
[19]: ## Plot the priors for `a` and `b` used in the Stan model.
data = (
    pd.DataFrame({"x": np.linspace(0, 10, 101)})
    .assign(y=lambda df: scipy.stats.norm(loc=0.5, scale=0.5).pdf(df["x"]))
)
(
    ggplot(data, aes("x", "y")) +
    geom_line()
)
```

```
[19]: <lets_plot.plot.core.PlotSpec at 0x7fe0308e1c50>
```

But the bottom row of the pairs plot further above shows that the values of b (β) around 0.5 have the highest $\text{lp}_{\text{--}}$ (log-probability) and that values of a (α) around 1.0 have the highest $\text{lp}_{\text{--}}$; 0.5 and 1.0 are indeed their true values. We often learn more when things go “wrong” and hopefully reading these diagnostic plots has helped you learn a little more about troubleshooting models that do not perfectly fit the noisy data!

1.4 ODE with a forcing function

1.4.1 Modified equations in Python

Finally! We get to discuss the main topic about how to use a forcing function when the equation of a state variable is unknown. Let’s say that we did not know the equation terms for the predator state variable, y . Then, instead of two ODEs, we need to solve a single ODE. But we must replace the y term in the x equation with a fixed trajectory of y .

Before we had:

$$\begin{aligned}\frac{dx}{dt} &= \alpha x - \beta xy \\ \frac{dy}{dt} &= -\gamma y + \delta xy\end{aligned}$$

Now we have only the first equation and pretend we do not yet know the process that governs the second equation:

$$\frac{dx}{dt} = \alpha x - \beta xy$$

We cannot fix y as a single value; y must vary in some way because x varies and y and x influence each other. So we must supply a fixed trajectory of y based on the noisy data that only depends on time because y is no longer part of the system of equations that the ODE solver has access to. We must define an external function that produces a fixed trajectory of y .

Ideally, we would use a spline function. Splines are wonderful mathematical functions because they're differentiable while still allowing us to fit complex, squiggly looking curves. However, spline functions are numerically harder to implement and Stan does not yet provide the mathematical primitives to easily add them. Therefore, we will use a simpler polynomial function:

$$\begin{aligned} \frac{dy}{dt} &= c_n t^n + c_{n-1} t^{n-1} \dots + c_2 t^2 + c_1 t^1 + c_0 t^0 \\ \Rightarrow \frac{dy}{dt} &= \sum_{i=0}^n c_i t^i \end{aligned}$$

Where, the polynomial terms are $t_0, t_1, t_2, \dots, t_{n-1}, t_n$ and their corresponding coefficients are $c_0, c_1, c_2, \dots, c_{n-1}, c_n$.

Note: Unlike a spline function, a polynomial function has trouble fitting periodic data. See the plot below.

```
[20]: def poly(x, degree):
    # Workaround statsmodels / patsy not supporting poly(variable, degree)
    # like R does. See https://stackoverflow.com/a/77567246
    return np.vander(x, degree + 1, increasing=True)[: , 1:]

max_time = 7
max_degree = len(times[times <= max_time]) - 2
fits_poly = []
for i in range(1, max_degree + 1):
    model = smf.ols(f"y ~ poly(time, {i})",
                    data=ode_to_df(noisy).loc[times <= max_time, :])
    fits_poly += [model.fit()]
```

```
/tmp/ipykernel_1423/1545647153.py:12: SingularMatrixWarning: The design matrix
is rank-deficient. The model parameters are not uniquely determined.
fits_poly += [model.fit()]
```

```
[21]: data = (
    ode_to_df(noisy)
    .loc[:, ["time"]]
    .query(f"time <= {max_time}")
```

```

.assign(**{
    re.findall(r"poly\[([^\,]+, (\d+))\]$",
               fits_poly[i].model.formula)[0]:
        #f"{fits_poly[i].degree}":
        # We have force evaluate i here by running a (lambda i: [...])(i)
        # returning a lambda, otherwise all the predictions will use the last
        # value of i of the loop, because evaluation of i in the returned
        # lambda is deferred until .assign() is run.
        (lambda i: lambda df: (
            fits_poly[i]
            #.fit_transform(
            .predict(
                df)))(i)
        for i in range(len(fits_poly))
    })
)
data = pd.melt(
    data,
    id_vars=["time"],
    var_name="degree",
    value_name="y",
)
data["degree"] = pd.to_numeric(data["degree"])
print(data)
(
    ggplot(data, aes("time", "y")) +
    facet_wrap("degree") +
    geom_line() +
    geom_point(data=ode_to_df(noisy).query(f"time <= {max_time}"))
)

```

	time	degree	y
0	0.0	1	6.175424
1	0.5	1	5.604266
2	1.0	1	5.033108
3	1.5	1	4.461951
4	2.0	1	3.890793
..	...	...	...
190	5.0	13	0.009289
191	5.5	13	-0.005261
192	6.0	13	0.001507
193	6.5	13	0.530631
194	7.0	13	0.107928

[195 rows x 3 columns]

[21]: <lets_plot.plot.core.PlotSpec at 0x7fe03041b850>

```
[22]: fits_poly[-1].summary()
```

```
[22]:
```

Dep. Variable:	y	R-squared:	1.000
Model:	OLS	Adj. R-squared:	0.999
Method:	Least Squares	F-statistic:	2139.
Date:	Wed, 23 Sep 2026	Prob (F-statistic):	0.000467
Time:	20:54:40	Log-Likelihood:	31.594
No. Observations:	15	AIC:	-37.19
Df Residuals:	2	BIC:	-27.98
Df Model:	12		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.4183	0.081	67.189	0.000	5.071	5.765
poly(time, 13)[0]	58.0522	3.377	17.191	0.003	43.523	72.582
poly(time, 13)[1]	-149.5375	13.726	-10.894	0.008	-208.598	-90.477
poly(time, 13)[2]	137.3511	17.801	7.716	0.016	60.758	213.944
poly(time, 13)[3]	-3.7654	3.514	-1.071	0.396	-18.886	11.355
poly(time, 13)[4]	-110.4549	13.991	-7.895	0.016	-170.653	-50.257
poly(time, 13)[5]	116.0801	16.699	6.951	0.020	44.231	187.929
poly(time, 13)[6]	-63.6977	9.669	-6.588	0.022	-105.302	-22.094
poly(time, 13)[7]	21.8934	3.425	6.392	0.024	7.156	36.630
poly(time, 13)[8]	-4.9451	0.789	-6.270	0.025	-8.338	-1.552
poly(time, 13)[9]	0.7350	0.119	6.186	0.025	0.224	1.246
poly(time, 13)[10]	-0.0693	0.011	-6.122	0.026	-0.118	-0.021
poly(time, 13)[11]	0.0038	0.001	6.069	0.026	0.001	0.006
poly(time, 13)[12]	-8.95e-05	1.49e-05	-6.023	0.026	-0.000	-2.56e-05

Omnibus:	1.874	Durbin-Watson:	3.777
Prob(Omnibus):	0.392	Jarque-Bera (JB):	0.318
Skew:	-0.007	Prob(JB):	0.853
Kurtosis:	3.713	Cond. No.	1.30e+15

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.3e+15. This might indicate that there are strong multicollinearity or other numerical problems.

```
[23]: fits_poly[0].pvalues.iloc[1]
```

```
[23]: np.float64(0.0016193213953972394)
```

```
[24]: ## Choose our polynomial degree.
criteria = pd.DataFrame(
    {
        "degree": [int(re.findall(r"poly\[([^\,]+\, (\d+)\)]$",
                                fit.model.formula)[0])
                    for fit in fits_poly],
        "aic": [fit.aic for fit in fits_poly],
```

```

        "bic": [fit.bic for fit in fits_poly],
    })
print(criteria)
degree = 3
assert degree == criteria.query("degree > 1").reset_index().query("aic.
    ↳argmax()").degree
assert degree == criteria.query("degree > 1").reset_index().query("bic.
    ↳argmax()").degree
(
    ggplot(criteria.melt(id_vars=["degree"]),
        aes("degree", "value", group="variable", color="variable")) +
    geom_line() +
    geom_vline(xintercept=degree, linetype="longdash")
)

```

	degree	aic	bic
0	1	70.825895	72.241996
1	2	64.625040	66.749190
2	3	66.575140	69.407341
3	4	65.097765	68.638016
4	5	57.120579	61.368880
5	6	39.349149	44.305500
6	7	27.635288	33.299689
7	8	3.484562	9.857014
8	9	-0.640615	6.439887
9	10	1.359079	9.147632
10	11	1.818656	10.315258
11	12	-19.181517	-9.976865
12	13	-37.187823	-27.983170

[24]: <lets_plot.plot.core.PlotSpec at 0x7fe030357e90>

```

[25]: # Now generate the ODE data using the forcing function (using t) and
# ignore any input for predator (dy_dt).
def y(t):
    """Return a positive scalar value for the predator y at time t.

    Importantly, this function is differentiable to help the ODE solver
    converge on a result. Well, almost; there are discontinuities to keep
    the output positive, but the solver seems to tolerate these.

    Parameters
    -----
    t : float
        Single timepoint at which to solve the differentiable equation.

    """

```

```

return np.maximum(
    fits_poly[degree]
    .predict(pd.DataFrame({"time": t}, index=[0]))
    .values,
    np.array([0]),
)

def lv_poly(t, y_, p=true_params):
    """Right-hand side of a Lotka-Volterra ODE with predator forcing function.

    Parameters
    -----
    t : float
        Single timepoint at which to solve the system of equations.
    y_ : array_like
        State variables at the timepoint `t`.
    p : dict of float, optional
        Model parameters.

    """
    # Unpack y into convenience variances that must match the return value of
    # this function.
    x = y_[0]

    # Decode the initial parameters and initial state variable values.
    p = Namespace(**p)

    # Encode the equation terms.
    prey_growth = p.a * x
    prey_death = p.b * x * y(t)

    # Return the equation RHS.
    dx_dt = prey_growth - prey_death
    return (dx_dt, )

```

```

[26]: fit_ode_ff = solve_ivp(fun=lv_poly,
                             t_span=[np.min(times), max_time],
                             # Preserve dimensions when subsetting to x by passing
                             # [1] as a list.
                             y0=list(noisy[1, [1]]),
                             t_eval=times[times <= max_time])

```

```

[27]: data = pd.concat(
    (
        ode_to_df(noisy)
        .query(f"time <= {max_time}")
    )

```

```

        .rename(columns=lambda col: col + "_orig" if col != "time" else col),
        ode_to_df(fit_ode_ff.y, times[times <= max_time]
        .reshape(-1, 1))
        .filter(["x"])
        .rename(columns={"x": "x_ff"}),
        pd.DataFrame({"y_ff": map(lambda t: y(t)[0], times[times <= max_time])})
    ),
    axis=1,
)
data = pd.melt(data, id_vars=["time"], var_name="variable_type")
data = pd.concat(
    (
        data.drop(columns=["variable_type"]),
        data.loc[:, "variable_type"]
        .str.split("_", expand=True)
        .rename(columns={0: "variable", 1: "type"}),
    ),
    axis=1)
data["type"] = pd.Categorical(data["type"],
                              categories=["orig", "ff"],
                              ordered=True)
(
    ggplot(data, aes("time", "value", group="variable", color="variable")) +
    facet_wrap("type", order=0) +
    geom_line() +
    geom_point(data=(
        data
        .query("type == 'orig'")
        .assign(type="ff")
    ))
)
)

```

[27]: <lets_plot.plot.core.PlotSpec at 0x7fe030713950>

1.4.2 Fit the ODE parameters using the equivalent modified Stan model

We implement these equations in the Stan code.

The model we are going to fit is saved in a separate lv-forcing-function.stan file that looks like this:

```
[28]: %stanf lv-forcing-function.stan model_lv_ff
```

```
INFO:cmdstanjupyter:Creating CmdStanPy model & assigning it to variable
"model_lv_ff"
```

```
20:54:40 - cmdstanpy - INFO - compiling stan file /__w/stan-ode-forcing-
function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function.stan to exe
file /__w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-
```

forcing-function

20:54:49 - cmdstanpy - INFO - compiled model executable: `__w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function`

20:54:49 - cmdstanpy - WARNING - Stan compiler has produced 1 warnings:

20:54:49 - cmdstanpy - WARNING -

--- Translating Stan model to C++ code ---

```
bin/stanc --filename-in-msg=lv-forcing-function.stan --o=__w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function.hpp
__w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function.stan
```

Warning in 'lv-forcing-function.stan', line 3, column 7 to column 8:

Functions do not need to be declared before definition; all user defined function names are always in scope regardless of definition order.

--- Compiling C++ code ---

```
g++ -std=c++17 -pthread -D_REENTRANT -Wno-sign-compare -Wno-ignored-attributes
-Wno-class-memaccess -isystem stan/lib/stan_math/lib/tbb_2020.3/include
-O3 -I src -I stan/src -I stan/lib/rapidjson_1.1.0/ -I lib/CLI11-1.9.1/ -I
stan/lib/stan_math/ -isystem stan/lib/stan_math/lib/eigen_5.0.1 -isystem
stan/lib/stan_math/lib/boost_1.87.0 -isystem
stan/lib/stan_math/lib/sundials_6.1.1/include -isystem
stan/lib/stan_math/lib/sundials_6.1.1/src/sundials -DBOOST_DISABLE_ASSERTS
-c -Wno-ignored-attributes -x c++ -o __w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function.o __w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function.hpp
```

--- Linking model ---

```
g++ -std=c++17 -pthread -D_REENTRANT -Wno-sign-compare -Wno-ignored-attributes
-Wno-class-memaccess -isystem stan/lib/stan_math/lib/tbb_2020.3/include
-O3 -I src -I stan/src -I stan/lib/rapidjson_1.1.0/ -I lib/CLI11-1.9.1/ -I
stan/lib/stan_math/ -isystem stan/lib/stan_math/lib/eigen_5.0.1 -isystem
stan/lib/stan_math/lib/boost_1.87.0 -isystem
stan/lib/stan_math/lib/sundials_6.1.1/include -isystem
stan/lib/stan_math/lib/sundials_6.1.1/src/sundials -DBOOST_DISABLE_ASSERTS
-Wl,-L,"/github/home/.cmdstan/cmdstan-2.40.0/stan/lib/stan_math/lib/tbb"
-Wl,-rpath,"/github/home/.cmdstan/cmdstan-2.40.0/stan/lib/stan_math/lib/tbb"
__w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function.o src/cmdstan/main.o -ltbb
stan/lib/stan_math/lib/sundials_6.1.1/lib/libsundials_nvecserial.a
stan/lib/stan_math/lib/sundials_6.1.1/lib/libsundials_cvodes.a
stan/lib/stan_math/lib/sundials_6.1.1/lib/libsundials_idas.a
stan/lib/stan_math/lib/sundials_6.1.1/lib/libsundials_kinsol.a
stan/lib/stan_math/lib/tbb/libtbb.so.2 -o __w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function
rm __w/stan-ode-forcing-function/stan-ode-forcing-function/stanodeffpy/lv-forcing-function.o __w/stan-ode-forcing-function/stan-ode-forcing-
```

function/stanodeffpy/lv-forcing-function.hpp

INFO:cmdstanjupyter:StanModel now available as variable "model_lv_ff"!

Compilation took 9 seconds.

```
functions {
  /* Forward declaration of prey forcing function. */
  real Y(real t, int N_coef, vector coef);

  /* Lotka-Volterra ODE equations with predator forcing function.

  See https://en.wikipedia.org/wiki/Lotka%E2%80%93Volterra_equations

  @param t The single time point at which to evaluate the ODE.
  @param x State variables of populations.
  @param a Prey growth rate parameter (alpha).
  @param b Prey death rate parameter from the presence of predators (beta).
  @return Left-hand side (LHS) of the system of ODE equations (prey only).
  */
  vector lv(real t,
            vector x,
            real a,
            real b,
            int N_coef,
            vector coef) {
    vector[1] dx;
    real y_ = Y(t, N_coef, coef);
    dx[1] = a * x[1] - b * x[1] * y_;
    return dx;
  }

  /* Predator forcing function.

  The original predator data is approximated as a continuous,
  differentiable function here using a polynomial because it's
  simple to implement in Stan; ideally one would want to use a more
  precise spline, but that's more difficult. Implementing a spline
  can be done later once the model is working.

  @param t Single time point at which to evaluate the forcing
  function.
  @return Predator value at time t.
  */
  real Y(real t, int N_coef, vector coef) {
    real predator = 0;
    for (i in 1:N_coef) {
      predator += coef[i] * pow(t, i-1);
    }
  }
}
```

```

    /* Set negative values to zero, otherwise the calculation will fail.*/
    if (predator < 0) {
        return 0;
    }
    return predator;
}
}

data {
    /* Dimensions. */
    int<lower=1> T;
    int<lower=1> V;

    /* Experimental data. */
    array[T] real ts;
    array[T] vector[V] y;

    /* Coefficients of fitted polynomial. */
    int<lower=1> N_coef;
    vector[N_coef] coef;

    /* Initial conditions. */
    real<lower=0> t0;
    vector<lower=0>[V] y0;
}

parameters {
    /* Prey death rate. */
    real<lower=0> b;
    /* Prey growth rate; must be higher than prey death rate. */
    real<lower=b> a;
}

model {
    array[T] vector[V] mu = ode_rk45(lv, y0, t0, ts, a, b, N_coef, coef);
    // Sample positive priors for all our ODE parameters; the T[0, ]
    // truncates the normal distrubiton at 0 so that negative values are
    // not sampled.
    a ~ normal(0.5, 0.5);
    b ~ normal(0.5, 0.5);
    // The state variables computed by the ODE should be normally
    // distributed.
    for (t in 2:T) {
        y[t] ~ normal(mu[t], 1);
    }
}

```

```
[29]: # Run Stan using the same forcing function.
fit_stan = model_lv_ff.sample(
    chains=4,
    parallel_chains=4,
    threads_per_chain=1,
    iter_warmup=1_000,
    iter_sampling=5_000,
    seed=12345,
    show_progress=False,
    data={
        # Must remove the row containing timepoint 0 and instead
        # provide it with the initial conditions.
        #
        # Dimensions:
        "T": len(times[(times > 0) & (times <= max_time)]),
        "V": noisy.T.shape[1] - 1,
        # Experimental data:
        "ts": times[(times > 0) & (times <= max_time)],
        "y": noisy.T[(times > 0) & (times <= max_time), 0].reshape(-1, 1),
        # Coefficients of fitted polynomial:
        "N_coef": len(fits_poly[degree].params.values),
        "coef": fits_poly[degree].params.values,
        # Initial conditions:
        "y0": noisy.T[0, 0],
        "t0": 0,
    },
)
```

```
20:54:49 - cmdstanpy - INFO - CmdStan start processing
20:54:49 - cmdstanpy - INFO - Chain [1] start processing
20:54:49 - cmdstanpy - INFO - Chain [3] start processing
20:54:49 - cmdstanpy - INFO - Chain [2] start processing
20:54:49 - cmdstanpy - INFO - Chain [4] start processing
20:55:02 - cmdstanpy - INFO - Chain [2] done processing
20:55:05 - cmdstanpy - INFO - Chain [4] done processing
20:55:06 - cmdstanpy - INFO - Chain [1] done processing
20:55:06 - cmdstanpy - INFO - Chain [3] done processing
20:55:06 - cmdstanpy - WARNING - Non-fatal error during sampling:
Exception: ode_rk45: ode parameters and data is inf, but must be finite! (in
'lv-forcing-function.stan', line 78, column 2 to column 71)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
Exception: ode_rk45: ode parameters and data is inf, but must be finite!
```



```

(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv-forcing-function.stan', line 78, column 2
to column 71)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be finite! (in
'lv-forcing-function.stan', line 87, column 4 to column 28)
Exception: normal_lpdf: Location parameter[1] is -nan, but must be finite! (in
'lv-forcing-function.stan', line 87, column 4 to column 28)
  Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv-forcing-function.stan', line 87, column 4 to column 28)
  Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv-forcing-function.stan', line 78, column 2
to column 71)
  Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv-forcing-function.stan', line 78, column 2
to column 71)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv-forcing-function.stan', line 78, column 2
to column 71)
Exception: ode_rk45: ode parameters and data is inf, but must be finite! (in
'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv-forcing-function.stan', line 87, column 4 to column 28)
  Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv-forcing-function.stan', line 87, column 4 to column 28)
  Exception: ode_rk45: Failed to integrate to next output time (0.5) in
less than max_num_steps steps (in 'lv-forcing-function.stan', line 78, column 2
to column 71)
  Exception: normal_lpdf: Location parameter[1] is -nan, but must be
finite! (in 'lv-forcing-function.stan', line 87, column 4 to column 28)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)
  Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)

```

Exception: ode_rk45: ode parameters and data is inf, but must be finite!
(in 'lv-forcing-function.stan', line 78, column 2 to column 71)

Exception: normal_lpdf: Location parameter[1] is -nan, but must be finite!
(in 'lv-forcing-function.stan', line 87, column 4 to column 28)
Consider re-running with show_console=True if the above output is unclear!

20:55:06 - cmdstanpy - WARNING - Some chains may have failed to converge.

Chain 1 had 5 divergent transitions (0.1%)

Use the "diagnose()" method on the CmdStanMCMC object to see further information.

1.4.3 Inspect the Stan forcing function fit

```
[30]: fit_stan.summary()
```

```
[30]:
```

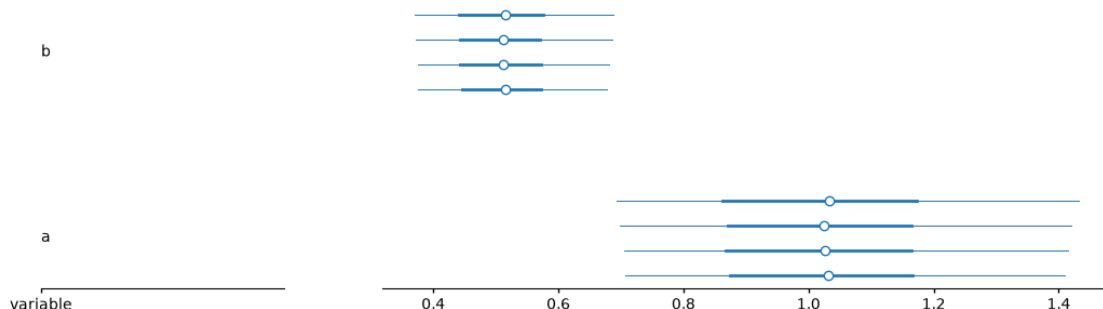
	Mean	MCSE	StdDev	MAD	5%	50%	95%	\
lp__	-3.866870	0.018318	1.074400	0.777238	-6.047220	-3.538210	-2.839280	
b	0.513535	0.001885	0.098303	0.095737	0.369901	0.503883	0.690478	
a	1.028470	0.004357	0.226977	0.219892	0.693115	1.007850	1.432250	

	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	4398.57	4224.69	115.0950	1.00044
b	2889.45	2613.90	75.6064	1.00067
a	2850.12	2634.85	74.5773	1.00084

```
[31]: ## Prepare data for plotting.  
idata_stan = az.from_cmdstanpy(fit_stan)
```

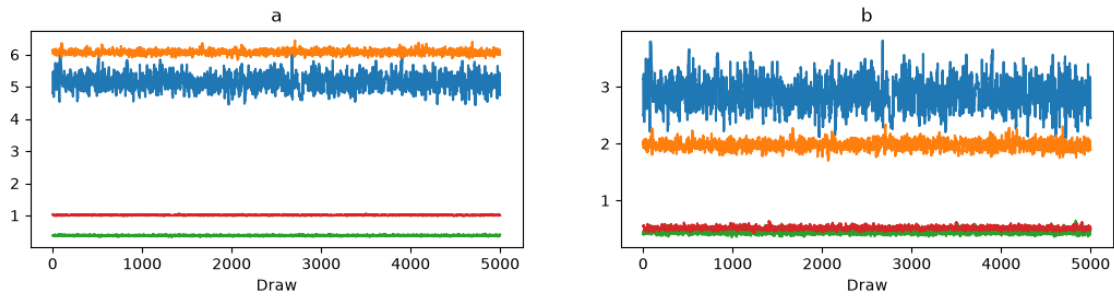
```
[32]: ## Parameter plot of the fits.  
azp.plot_forest(idata_stan)
```

```
[32]: <arviz_plots.plot_collection.PlotCollection at 0x7fe025ef9310>
```



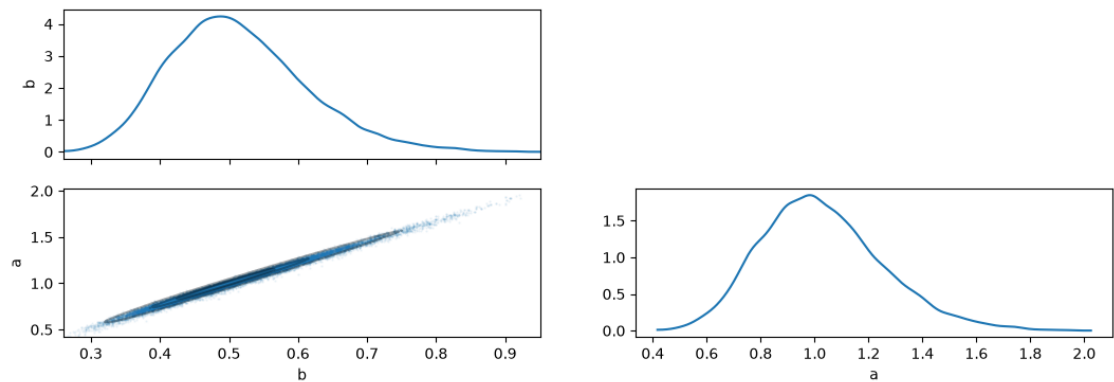
```
[33]: ## Show all samples after warmup.  
azp.plot_trace(idata1)
```

[33]: <arviz_plots.plot_collection.PlotCollection at 0x7fe03048da70>



```
[34]: ## Show the pairs plot.  
## FIXME: Possible to include lp_?  
azp.plot_pair(idata_stan,  
              visuals={  
                "contour": {"color": "black", "alpha": 0.3},  
                "scatter": {"size": 2, "alpha": 0.1},  
              })
```

[34]: <arviz_plots.plot_matrix.PlotMatrix at 0x7fe030202490>



```
[35]: ## Plot the ODEs using a sample of 100 parameters from each of the chains.  
df_fit_stan = fit_stan.draws_pd()  
## Sample greater variation by only choosing one sample from each qcut n-tile.  
qcut_20 = lambda x: pd.qcut(x, np.linspace(0, 1, 20), labels=False)  
grouped = (  
    df_fit_stan  
    .filter(["chain__", "iter__", "a", "b"])  
    .groupby("chain__")  
)
```

```

n = 100
prng = np.random.Generator(np.random.MT19937(seed=123))
df_fit_stan_subset = (
    df_fit_stan.join(
        pd.DataFrame({
            "qa": grouped["a"].transform(qcut_20),
            "qb": grouped["b"].transform(qcut_20),
        })
    )
    .groupby(["chain__", "qa", "qb"])
    .sample(n=1, random_state=prng)
    ## Now sample the 100 parameters from each chain.
    .groupby("chain__")
    .sample(n=n, replace=True)
    .reset_index()
    .reset_index(names="sample")
)
n_node = df_fit_stan_subset.shape[0]
arr = np.empty(np.array(pristine.y.T[times <= max_time].shape) * np.
    ↪array([n_node, 1]),
                dtype=pristine.y.dtype)
arr_y = np.array([list(map(lambda t: y(t)[0], times[times <= max_time]))]).T
for i in range(n_node):
    ode_x = solve_ivp(fun=lv_poly,
                      t_span=[np.min(times), max_time],
                      y0=noisy.T[0, [0]],
                      t_eval=times[times <= max_time],
                      args=[
                          {
                              "a": df_fit_stan_subset.loc[i, "a"],
                              "b": df_fit_stan_subset.loc[i, "b"],
                          }
                      ])
    assert ode_x.success
    begin = i * len(times[times <= max_time])
    end = (i + 1) * len(times[times <= max_time])
    arr[begin:end, [0]] = ode_x.y.T
    arr[begin:end, [1]] = arr_y

data = (
    pd.concat(
        (
            pd.DataFrame(
                {
                    "sample": np.repeat(range(n_node), len(times[times <=
    ↪max_time])),
                    "time": np.tile(times[times <= max_time], n_node),

```

```

    }
    ),
    pd.DataFrame(arr, columns=true_y0.keys())
  ),
  axis=1
)
.join(df_fit_stan_subset.filter(["sample", "chain__"]),
      on="sample",
      lsuffix="l",
      rsuffix="r")
)
data = pd.melt(data, id_vars=["sample", "time", "chain__", "samplel", "sampler"],
               value_vars=["x", "y"])
data

```

```

[35]:
   sample  time  chain__  samplel  sampler  variable  value
0         0   0.0      1.0         0         0         x  9.929955
1         0   0.5      1.0         0         0         x  2.731218
2         0   1.0      1.0         0         0         x  0.870850
3         0   1.5      1.0         0         0         x  0.375425
4         0   2.0      1.0         0         0         x  0.233646
...
11995    399   5.0      4.0        399        399         y  0.025411
11996    399   5.5      4.0        399        399         y  0.600875
11997    399   6.0      4.0        399        399         y  0.971245
11998    399   6.5      4.0        399        399         y  0.754879
11999    399   7.0      4.0        399        399         y  0.000000

```

[12000 rows x 7 columns]

```

[36]: (
  ggplot(
    data,
    aes(
      "time",
      "value",
      group=["sample", "variable"],
      color="variable",
    )
  ) +
  facet_wrap("chain__") +
  geom_line(alpha=0.1) +
  geom_point(
    data=(
      pd.melt(
        ode_to_df(noisy)
        .query(f"time <= {max_time}"),

```

```

        id_vars=["time"],
    ).assign(sample=0)
    # The sample=0 is never used anywhere, but the plot will error out
    # without it.
),
mapping=aes(
    group=[],
),
)
)

```

[36]: <lets_plot.plot.core.PlotSpec at 0x7fe025ed56d0>

1.5 Finishing thoughts

1. Don't simply choose a forcing function only based on how it fits its own data; instead, choose a forcing function by how well the other equations fit their data by running the full ODE system.
2. Spline functions are great as forcing functions, but not always practical to use. Polynomials can work, but may not work well with some types of periodic data; subset the data to timepoints where the fit looks good.
3. You may have noticed `seed=...` used in a few places. This is to make the stochastic code outputs reproducible, but one would typically not use these; it is more common to use store the value of `np.random.get_state()` to record a seed used for a simulation than to force a specific outcome because well-adjusted simulations should produce very similar results in spite of pseudo-random number generator initial states.